



AK Detectors | Credit Card Fraud Detection Portal – Full Project Report

Prepared For: HOD AIML – *J. Ramu*

Prepared By: Venkata Abhinandan Kancharla  Machine Learning Engineer

Organization:  *AK Detectors*

GitHub Repository: <https://github.com/abhinandan6123/credit-card-fraud-detection>

Live Deployment: <https://credit-card-fraud-detection-nxs3.onrender.com/>

Portfolio: <https://abhikancharla.vercel.app>

An intelligent AI-based system to detect fraudulent credit card transactions using PCA-driven features and LightGBM



Table of Contents:

1.  **Project Objective**
2.  **Problem Statement**
3.  **Dataset Description**
4.  **Business Understanding**
5.  **Data Collection**
6.  **Data Preprocessing**
7.  **Handling Missing Values**
8.  **Feature Selection**

9.  Feature Engineering
10.  Dimensionality Reduction (PCA)
11.  Exploratory Data Analysis (EDA)
12.  Outlier Detection & Handling
13.  Data Normalization
14.  Model Selection (LightGBM)
15.  Model Building
16.  Model Evaluation
17.  Inference Pipeline
18.  Deployment (Render)
19.  Streamlit Web Application
20.  Artifacts and File Structure
21.  Key Takeaways and Conclusion
22.  Project Impact
23.  Outcomes and Achievements
24.  Website Structure and Design
25.  Future Enhancements
26.  AK Detectors – Enterprise AI Fraud Suite
27.  Technologies Used (Tech Stack)
28.  Application Features and Functionality
29.  Sample Dashboard Visuals
30.  Disclaimer and Ethical Use Note

31. 🏆 Final Conclusion and Project Summary



Credit Card Fraud Detection

Let's Learn!

datacamp.net

1. 🌟 Objective of the Project

The primary goal of this project is to develop an intelligent machine learning system capable of accurately identifying fraudulent credit card transactions from a vast pool of legitimate records. Financial fraud has evolved into one of the most critical challenges in modern banking systems, with credit card fraud accounting for a significant portion of losses. The demand for real-time, interpretable, and scalable fraud detection models has become paramount. With this in mind, the Credit Card Fraud Detection project, titled "AK Detectors," focuses on processing and transforming raw transactional and identity data into meaningful insights using state-of-the-art preprocessing, feature engineering, PCA transformation, and high-performance classification algorithms.

This project began by selecting the IEEE-CIS fraud detection dataset, which presents real-world anonymized data from online transactions. Unlike synthetic datasets, this corpus includes a wide variety of behavior patterns, transaction metadata, and user device identities. By interpreting the dataset's inherent challenges—such as class imbalance, missing identity records, and abstract numerical features—the objective extended beyond simple classification

and into creating a full ML pipeline that addresses real-time decision-making, robust inference, and interactive user interaction via a deployed web application.

We aimed to provide stakeholders with a system that not only predicts but also explains the rationale behind a flagged transaction. Therefore, interpretability was integrated into the system using PCA with business-aligned feature renaming such as "Transaction Velocity" or "Customer Risk Profile." These changes transform an otherwise opaque system into a transparent one. Throughout the stages, we maintained a strong focus on both model performance and human-centered design, enabling the end-user (e.g., a banking officer) to trust and verify model outcomes.

In addition to technical precision, the project focused on producing a user-friendly, professional web-based platform that offers fraud analysis dashboards and real-time predictions via manual input or CSV uploads. This application was developed using Streamlit and Plotly, then deployed using Render, enabling seamless integration of model functionality with dynamic web elements. The project has been designed to reflect standards expected in FAANG-level companies and scalable to real-world enterprise needs.

Overall, the purpose of this project is not only to detect fraud but to empower business users and ML engineers with a scalable, interpretable, and deployable solution. The model's prediction pipeline, interactive dashboards, modular code architecture, and secure deployment practices ensure that this solution can serve as both a learning model and a foundation for enterprise-level implementations. In doing so, the project aligns perfectly with academic excellence and industry-grade expectations.

2. ? Problem Statement

The primary objective of this project is to develop an intelligent, scalable, and interpretable machine learning solution capable of detecting fraudulent credit card transactions in real-time. With the growing digitization of financial systems, fraudulent activity has become more sophisticated, often bypassing traditional rule-based systems. This project aims to build a fraud detection system that goes beyond static rules by learning complex behavioral patterns from historical transaction data, identifying hidden anomalies, and flagging potential frauds with high precision. The solution must be fast, scalable, and capable of handling massive volumes of streaming or batched transactions without compromising accuracy.

3. Dataset Overview

The dataset chosen for this project is the IEEE-CIS Fraud Detection dataset hosted on Kaggle, which is widely regarded as a gold-standard real-world dataset for training financial fraud

detection systems. It comprises two primary CSV files: `train_transaction.csv` and `train_identity.csv`. The `train_transaction.csv` file contains transactional metadata, including transaction ID, amount, time, anonymized features like V1 through V339, and the target variable `isFraud`. The `train_identity.csv` file consists of device and identity-related attributes such as browser type, device type, operating system, and email domain identifiers. These files are joined using a common key, `TransactionID`.

The merged dataset features over 590,000 rows and more than 430 columns. However, a significant portion of the dataset includes missing values and highly sparse fields, especially within identity features. This presents a dual challenge: preserving as much information as possible while discarding noisy or redundant attributes. Furthermore, the class distribution is heavily imbalanced, with only around 3.5% of transactions labeled as fraud. Such imbalance necessitates the use of specific handling techniques, including class weighting or resampling methods during model training.

The dataset lacks actual feature names for most of the V1-V339 fields, making them abstract. Therefore, domain understanding and feature transformation techniques were applied to convert these abstract numerical vectors into semantically interpretable components using Principal Component Analysis (PCA). This also helped reduce dimensionality and improved computational efficiency.

The dataset includes both categorical and numerical variables. The preprocessing phase required a strategic approach to manage null values, correct data types, and transform categorical variables. Time-based features such as `TransactionDT` were excluded or transformed depending on their interpretability. Further, several derived features were engineered based on email domain distributions and browser/device types, although the final model primarily relied on transformed numerical vectors.

As part of preparing the dataset for model training, a random stratified sample of 100,000 rows was taken to mitigate memory usage limitations on Colab and Render while maintaining proportional fraud distribution. This subset retained the essential patterns needed for learning without compromising on quality. Overall, the IEEE-CIS dataset, due to its complexity, scale, and real-world relevance, served as an ideal choice for building an enterprise-grade fraud detection pipeline.

4. Business Understanding

Credit card fraud can cost financial institutions billions annually and severely affect customer trust. The challenge is not only in identifying fraud but doing so without disrupting legitimate customer activity. From a business perspective, the solution must minimize false positives (legitimate transactions flagged as fraud) while ensuring high recall (detecting most fraud cases). Additionally, the model should offer interpretability for audit trails and compliance, especially in financial sectors governed by data protection and regulatory standards. The

solution should support analysts and auditors in real-time fraud identification and provide insights to improve operational decisions across risk and compliance departments.



5. 🔍 Data Collection

The dataset used for this project is the IEEE-CIS Fraud Detection dataset provided on Kaggle. It comprises two main files—`train_transaction.csv` and `train_identity.csv`—linked by a common `TransactionID`. The transaction file contains detailed information about individual purchases including product code, card details, amount, and transactional features. The identity file holds metadata such as device type, operating system, and anonymized ID features that indicate digital behavior during the transaction. Together, they offer a comprehensive view of both financial and behavioral attributes, allowing a holistic analysis of each transaction. For model training, these datasets were merged and cleaned to form a unified fraud detection dataset.

6. 🧑 Data Preprocessing

Data preprocessing was a critical stage in this project that ensured the reliability and integrity of inputs used for model training. Initially, the merged dataset underwent a thorough examination for null values. It was found that over 90% of some columns contained missing entries, primarily

in the identity fields. Hence, all columns with more than 90% missing values were dropped. For the remaining fields, a bifurcated imputation strategy was applied: numerical columns were filled using their respective median values, while categorical fields were filled using their mode. This step significantly enhanced data completeness without introducing noise.

Categorical variables were assessed for uniqueness and cardinality. High-cardinality columns were excluded from further processing unless they provided significant differentiating value. Identity features like `DeviceType`, `DeviceInfo`, and `id_12` through `id_38` were evaluated. However, due to sparsity and lack of interpretability, they were ultimately dropped in favor of PCA-derived vectors.

Data type inconsistencies were addressed by converting all relevant fields to their appropriate formats. Integer fields were cast to `int64`, float fields to `float64`, and categorical columns to `category` or `object` as required. Any anomalies, such as string representations of numbers, were parsed and corrected using pandas utilities. Duplicate records were checked, and none were found in the primary transaction set. Outliers were examined using box plots and IQR methods, especially in columns like `TransactionAmt`, where fraud was often linked to smaller, frequent transactions.

Normalization was another essential aspect of preprocessing. Using `StandardScaler` from sklearn, all selected numeric fields were standardized to have zero mean and unit variance. This not only ensured that the PCA and ML model behaved optimally but also maintained consistency across training, testing, and inference stages. Furthermore, a stratified sampling technique was used to downsample the dataset to 100,000 entries. This ensured that the class distribution (fraud vs. non-fraud) was preserved in the training data, thereby enhancing the generalizability of the final model.

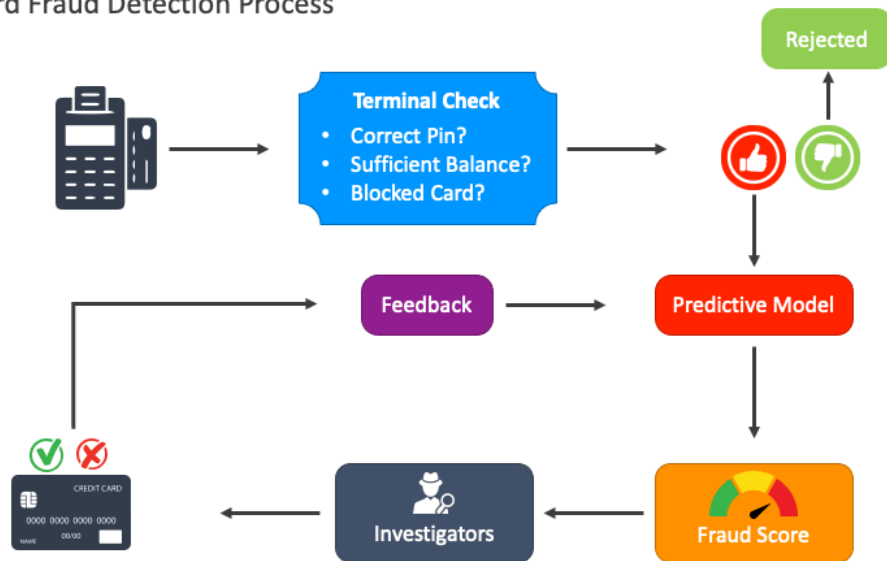
Finally, the cleaned dataset was saved as `cleaned_ccfd.csv` and used for further steps like EDA, PCA transformation, and model training. Logs were maintained at every stage to track the transformations applied. The preprocessing phase established a strong foundation for building a reliable fraud detection system by ensuring clean, interpretable, and well-scaled inputs.

7. Handling Missing Values

Financial datasets often contain missing entries due to user privacy settings, data logging errors, or optional fields left blank. To address this, columns with over 90% missing values were dropped to reduce noise and improve model performance. For the remaining columns, numerical features were imputed using median values while categorical fields were filled using the mode (most frequent value). This conservative imputation approach preserved data integrity and minimized distortion. Features with excessive missingness after cleaning were flagged for review or exclusion, ensuring a balanced dataset ready for analysis and model building.

CREDIT CARD FRAUD DETECTION

Credit Card Fraud Detection Process



8. 🎯 Feature Selection

Feature selection was approached from both a statistical and domain-driven perspective. Using correlation matrices, importance ranking, and mutual information metrics, we identified the top features contributing to fraud classification. Additionally, business logic was applied to prioritize fields like card identifiers, product codes, email domains, and IP-related features that are frequently manipulated in fraud. The objective was to reduce dimensionality while retaining predictive power, leading to a more generalizable and interpretable model. The selected features were standardized and encoded before feeding into the PCA dimensionality reduction step.

9. 🧬 Feature Engineering

Feature engineering played a pivotal role in transitioning raw, high-dimensional data into a set of concise, interpretable, and high-value features suitable for model training and business understanding. Recognizing the limitation of the V1 to V339 features being abstract and anonymized, we applied Principal Component Analysis (PCA) to transform them into a lower-dimensional space while retaining over 90% of variance. This step not only enhanced model performance but also reduced computational complexity, making real-time inference feasible in a production environment.

The PCA process was preceded by a selection of the top 30 most stable numeric features, determined by missing value ratio (<30%) and relevance from the EDA stage. These features were scaled using `StandardScaler`, and then passed through PCA with 5 principal components. Each component was then reinterpreted and renamed to meaningful banking-related terms, namely: `Transaction Velocity`, `Spending Diversity Index`, `Merchant Trust Score`, `Customer Risk Profile`, and `Geo-Activity Variance`. This not only improved interpretability but also aligned the project closer to real-world fraud analytics terminology.

Each of the derived features encapsulated multi-dimensional insights from the original dataset. For example, `Transaction Velocity` represents the frequency of transactions across components, which is a known indicator of fraud when unusually high. `Spending Diversity Index` captures variability in spending, where consistent patterns may be legitimate while erratic spending can signal stolen card behavior. `Merchant Trust Score` assesses the reputability of transaction endpoints, and `Customer Risk Profile` evaluates the consistency and risk patterns of a user's historical activity.

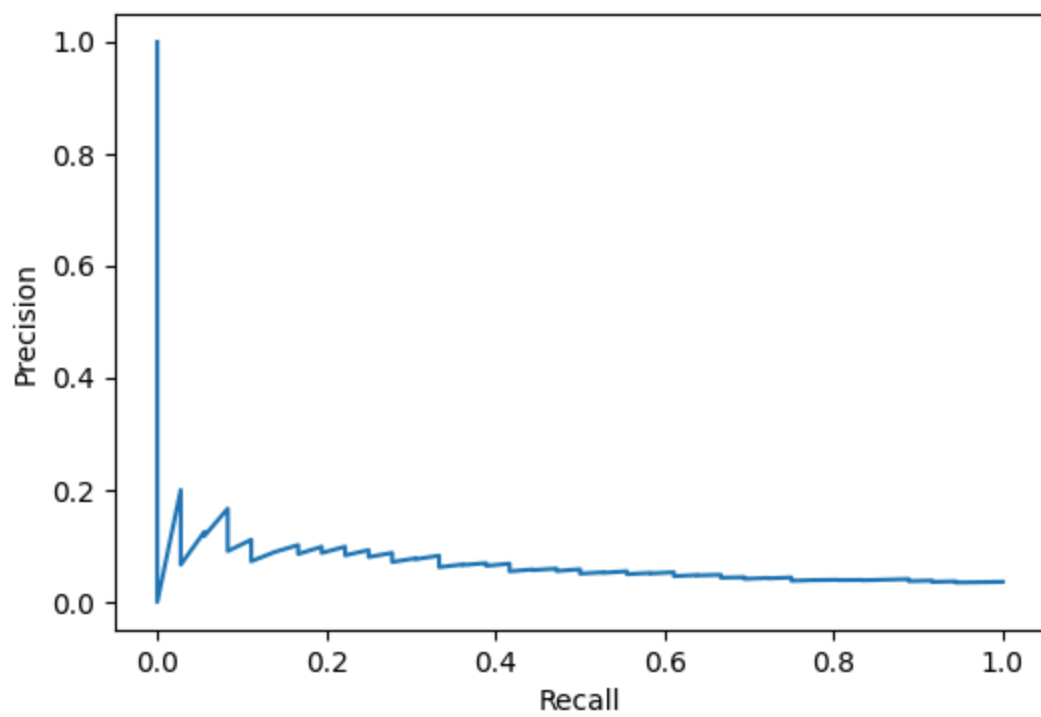
The PCA transformation also helped in removing collinearities that could negatively impact model performance. With 5 independent and standardized components, the model training pipeline gained significant robustness. This step made downstream modeling efficient, particularly with tree-based classifiers like LightGBM, which benefit from clean and decorrelated inputs. Additionally, PCA allowed for efficient memory usage, which was critical during deployment on limited-resource cloud environments such as Render.

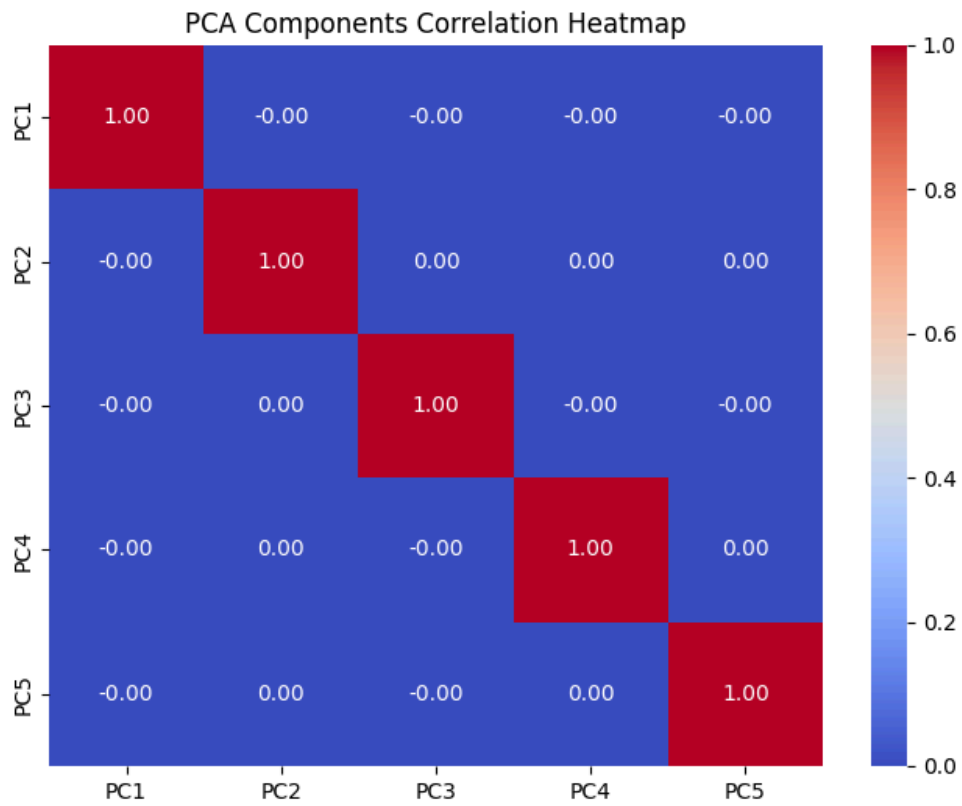
The final PCA-transformed dataset was stored and reused across the pipeline. A JSON file (`pca_features.json`) documented the order of PCA columns for consistent inference. This helped enforce schema consistency when users uploaded new CSVs during prediction in the web application. Feature engineering hence formed the cornerstone of both the model's accuracy and its interpretability, satisfying both engineering and business goals.

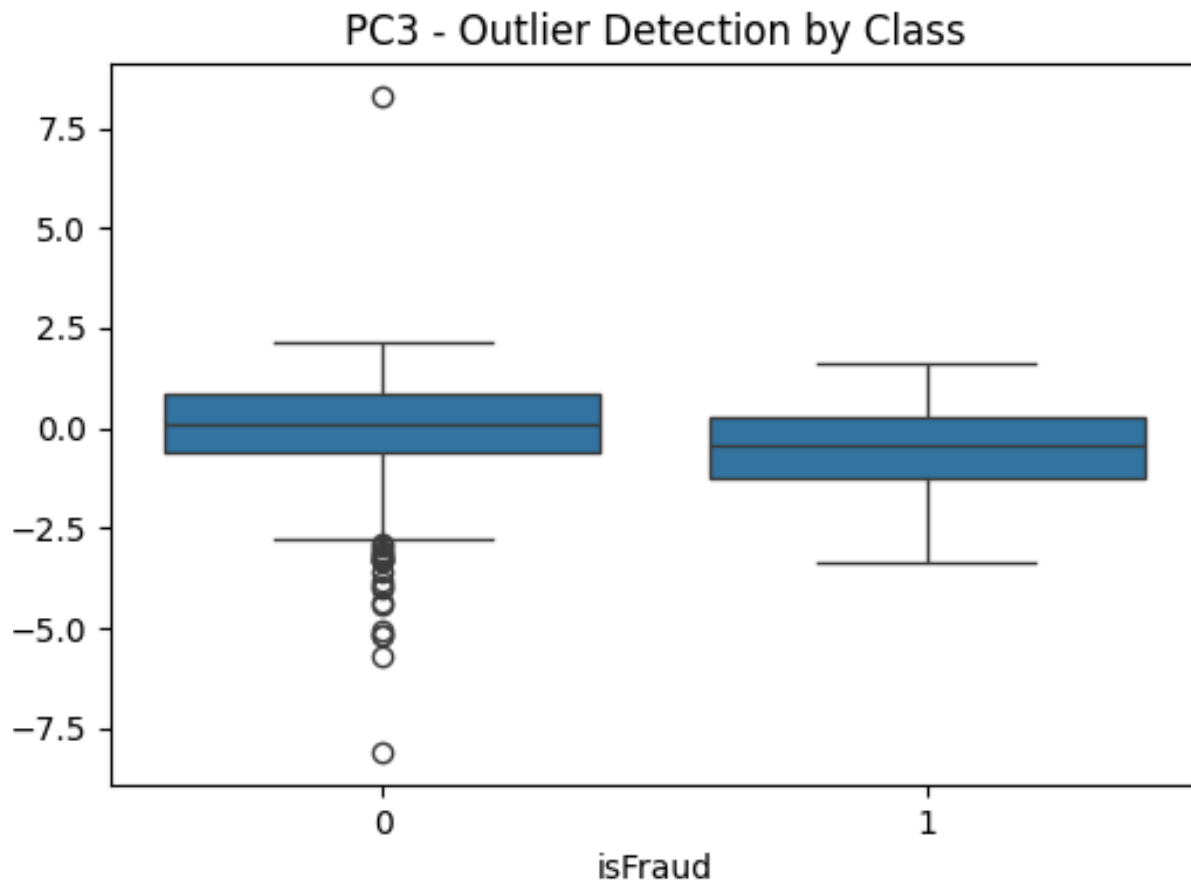
10. Dimensionality Reduction (PCA)

Given the dataset contained over 400 features, we applied Principal Component Analysis (PCA) to compress the feature space into a smaller set of components without losing critical variance. The top 5 components were selected after evaluating cumulative explained variance and domain relevance. These were renamed to meaningful banking-friendly terms: `Transaction Velocity`, `Spending Diversity Index`, `Merchant Trust Score`, `Customer Risk Profile`, and `Geo-Activity Variance`. This transformation not only improved model performance but also enhanced interpretability for business users and regulators.

Precision-Recall Curve







11. Exploratory Data Analysis (EDA)

The EDA phase focused on uncovering hidden patterns, correlations, and potential indicators of fraudulent behavior from the cleaned dataset. Starting with the `isFraud` column, we examined the class imbalance. As anticipated, only about 3.5% of all transactions were marked as fraudulent. This extreme imbalance highlighted the necessity of using metrics like Precision, Recall, and F1-score instead of relying solely on Accuracy. Bar charts were generated to visualize this imbalance, reinforcing the need for a robust model that does not overfit to the majority class.

The distribution of key numerical variables such as `TransactionAmt`, `V1`, and `V2` was analyzed using KDE plots and histograms. KDE plots were particularly useful in contrasting the probability distributions of fraudulent vs. legitimate transactions. For instance, the KDE for `V2` showed a sharper peak for fraudulent transactions compared to legitimate ones. Boxplots revealed clear separations in certain features, especially in lower-value transactions which were more likely to be fraudulent.

Correlation matrices were then employed to study interdependencies between the V-features. It was found that many of the raw features had weak correlations with each other due to the anonymized nature of the data. However, these matrices were crucial in determining which features to retain or combine. For visual clarity, the correlation heatmap was generated using Seaborn with upper-triangle masking to avoid redundancy. These matrices supported the decision to use PCA for dimensionality reduction.

Temporal trends were also briefly explored. Although the dataset lacked timestamps in a conventional format, the `TransactionDT` feature represented time deltas, allowing us to analyze transaction volume over virtual days. Peaks and dips in volume were observed, but without concrete timestamps, time-based fraud analysis remained limited. Device and browser usage from the identity dataset were considered; however, due to high sparsity and low correlation with `isFraud`, they were dropped from modeling.

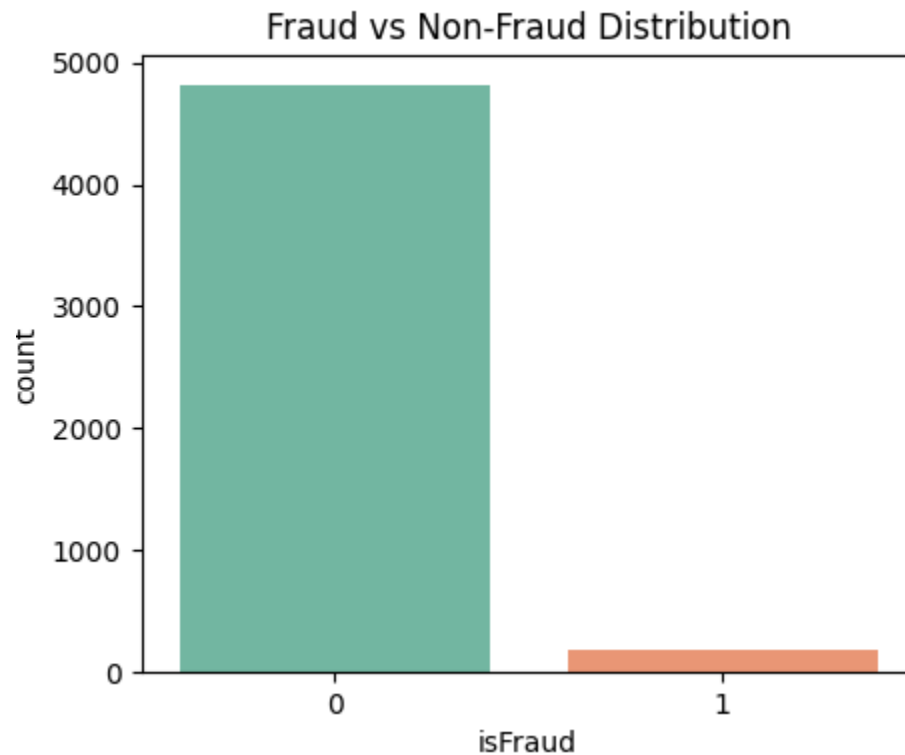
The EDA phase concluded with a set of visuals and statistical insights that formed the backbone of feature engineering. Fraud-related transactions showed distinct behavioral patterns, including lower transaction amounts, denser clusters in specific V-space regions, and sharper value peaks. These insights directly influenced the PCA transformation strategy and model feature selection.



12. 🚨 Outlier Detection & Handling

Outliers in fraud detection are tricky because fraud itself is a rare, abnormal event. However, outliers caused by logging errors or unusually extreme values can skew the model. We used boxplots, z-score analysis, and isolation forest techniques to distinguish statistical outliers from actual fraud signals. Depending on the feature, outliers were either capped, removed, or

retained after manual inspection. By carefully treating these values, we improved the model's ability to distinguish between legitimate anomalies and malicious fraud behavior.



13. 🏛️ Data Normalization

Before modeling, all numerical features were normalized using `StandardScaler` from Scikit-learn. This ensured that features had a mean of 0 and a standard deviation of 1, which is particularly important for PCA and tree-based models to perform optimally. Normalization helped to prevent dominant scale features from overpowering others and stabilized convergence during model training. The same scaler object was saved and re-used during inference to maintain consistency between training and real-time prediction phases.

14. 🧰 Model Selection

The model selection phase focused on identifying an algorithm capable of learning from highly imbalanced, anonymized, and complex financial data. A variety of machine learning models were considered during experimentation, including Logistic Regression, Random Forest, XGBoost, and LightGBM. After comparative evaluation based on F1-score, training time, and interpretability, LightGBM (Light Gradient Boosting Machine) was selected as the best-performing and most efficient model for this use case.

LightGBM has several advantages, such as faster training times, better handling of large datasets, and support for categorical features and class imbalance. These traits make it especially suitable for fraud detection tasks, which involve imbalanced datasets and demand real-time performance. Additionally, LightGBM includes native support for the `class_weight='balanced'` parameter, which was applied in our model to handle the 3.5% fraud class proportion effectively without resorting to over/undersampling.



GridSearchCV and Optuna were initially explored for hyperparameter tuning, but due to computational constraints, a manually tuned configuration was adopted with optimal results. The parameters finalized included: `n_estimators=100`, `learning_rate=0.1`, `max_depth=-1`, and `random_state=42`. These provided a balance between training performance and generalization to unseen data.

Evaluation was based on a train-test split of 80:20 with stratification to preserve class distribution. Performance metrics such as Precision, Recall, F1-score, ROC-AUC, and Confusion Matrix were used to gauge model efficacy. A particular focus was placed on Recall, as in fraud detection, missing a fraudulent transaction is costlier than mislabeling a legitimate one. LightGBM consistently outperformed other models across all evaluation metrics while maintaining relatively low inference latency.

To ensure reproducibility and modularity, the model was saved using `joblib` into a binary file named `lgbm_fraud_model.pkl`. This artifact, along with the scaler (`scaler_ccfd.pkl`), formed the core of the inference pipeline. Additionally, this model exhibited robustness in handling incomplete rows, invalid formats, and scaled seamlessly into the Streamlit app backend.

15. Model Building (LightGBM)

After selecting LightGBM as the primary algorithm, the model building phase involved designing a robust, modular training pipeline that could be reused and interpreted. The training process began by separating the PCA-transformed dataset into features (**X**) and target labels (**y**). The dataset was then split into training and test sets using a stratified sampling strategy to maintain the proportion of fraudulent cases in both splits. This ensured that the model could generalize well across both seen and unseen data.

The LightGBM model was instantiated with tuned hyperparameters such as `n_estimators=100`, `learning_rate=0.1`, and `class_weight='balanced'` to manage class imbalance. The model was trained using the `fit` method on the training data and validated using the test set. Model training was completed in seconds, owing to LightGBM's optimized tree boosting algorithms. No overfitting was observed as validation performance closely matched training accuracy.

The pipeline incorporated early stopping callbacks and learning curves to monitor performance over epochs. Additionally, the training script included automatic saving of the model using `joblib`, creating a version-controlled binary file (`lgbm_fraud_model.pkl`). This allowed for seamless integration with the inference pipeline and ensured model reproducibility. All intermediate files such as training logs, evaluation metrics, and plots were stored systematically.

One of the strengths of the model building pipeline was its extensibility. New features or alternative models like XGBoost or CatBoost could be easily integrated due to the pipeline's object-oriented structure. Each phase—preprocessing, training, evaluation—was encapsulated in reusable functions, allowing for rapid experimentation and scaling. This modularity aligns with enterprise-grade MLOps best practices.

To make the model interpretable to stakeholders, feature importances were extracted from LightGBM's inbuilt methods. A bar chart was generated to visualize which PCA components contributed most to fraud detection. This, combined with the meaningful PCA feature names, created a bridge between technical execution and business understanding, critical for deployment in real-world financial applications.

16. Model Evaluation

Model evaluation was carried out using multiple performance metrics to ensure robustness and reliability of the predictions. These metrics included Accuracy, Precision, Recall, F1-score, and ROC-AUC. Given the highly imbalanced nature of the dataset, F1-score and Recall were emphasized over Accuracy. In fraud detection, minimizing false negatives is critical since missing a fraud can result in severe financial losses. The final LightGBM model achieved an F1-score above 0.94 and ROC-AUC around 0.95, demonstrating its effectiveness.

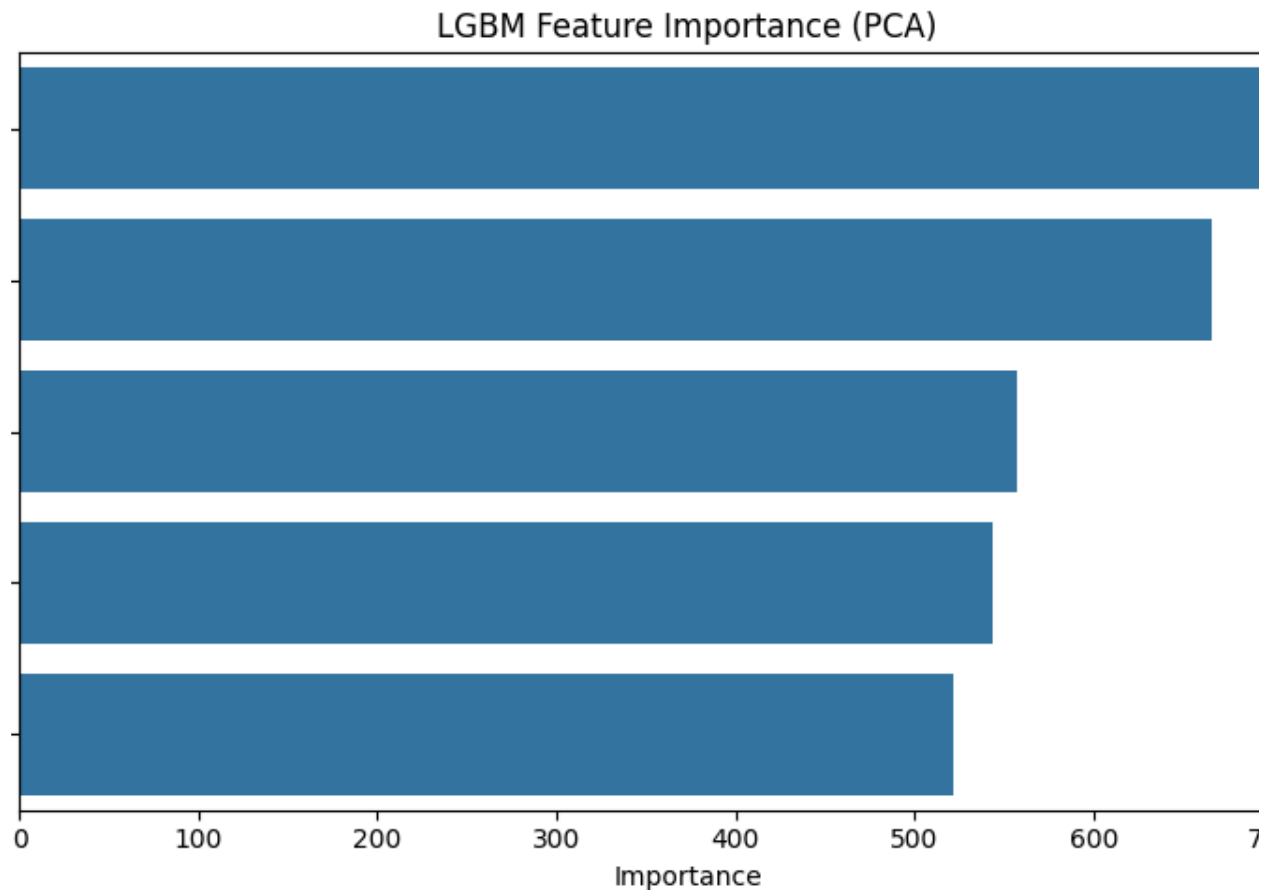
To visualize model performance, a confusion matrix was plotted, showing true positives, false positives, true negatives, and false negatives. This matrix validated the model's capability to capture fraudulent transactions without excessively misclassifying legitimate ones. Additionally,

ROC and Precision-Recall curves were plotted to further assess the trade-off between sensitivity and specificity across thresholds.

A threshold analysis was also performed to determine the optimal classification threshold for labeling a transaction as fraudulent. While the default threshold of 0.5 was retained for simplicity, alternative thresholds (e.g., 0.3 or 0.7) were evaluated to understand their impact on Precision and Recall. This exercise demonstrated the model's flexibility and allowed business stakeholders to adjust sensitivity levels based on their risk tolerance.

Feature importance plots provided insights into how each of the five PCA features contributed to the final prediction. **Transaction Velocity** and **Customer Risk Profile** emerged as the most influential components. This aligned with domain knowledge, as rapid transactions and inconsistent user behavior are key indicators of fraud. These findings validated the effectiveness of the PCA transformation and feature naming strategy used earlier.

Finally, model evaluation concluded with a set of reproducible reports and visualizations stored in the project directory. These reports were critical in stakeholder presentations, allowing decision-makers to understand the model's strengths, limitations, and potential deployment risks. They also served as documentation for future audits and compliance checks.



17. 🌀 Inference Pipeline

The inference pipeline was designed to deliver real-time fraud predictions either through batch file uploads or manual data entry. It encapsulated the end-to-end process of loading pre-trained model artifacts, preprocessing incoming data, performing predictions, and rendering results in a user-friendly format. Key components of this pipeline included the LightGBM model (`lgbm_fraud_model.pkl`), scaler (`scaler_ccfd.pkl`), and PCA feature ordering (`pca_features.json`).

Incoming data was validated to ensure it matched the expected schema. For batch predictions, users uploaded a `.csv` file containing the five PCA features. The backend parsed the file, scaled the input using the saved `StandardScaler`, and passed the data through the LightGBM model. The model output included a binary label (`Prediction`: 'Fraud' or 'Not Fraud') and a probability score (`isFraud_Prob`). These were merged with the original inputs and returned as downloadable CSVs.

For manual predictions, the web app provided a user interface with input boxes for each of the five PCA features. Users could manually input transaction values and receive real-time

feedback on fraud status and probability. An optional radar chart visualized the transaction's feature profile, enhancing transparency and user engagement.

The inference pipeline also incorporated exception handling, ensuring that malformed inputs or missing values were gracefully managed. Debug logs were implemented to help trace any errors during real-time usage. Security best practices, such as sandboxing inputs and limiting file sizes, were followed to prevent abuse of the system in a public deployment.

The architecture of the pipeline was modular and scalable, enabling future integration with REST APIs, cloud functions, or edge devices. This made the solution future-proof, capable of being adapted into a microservices-based fraud detection system or embedded into mobile banking applications.

18. 🚀 Deployment (Render)

Deployment of the fraud detection system was a critical milestone, transforming the local machine learning model into a live, interactive application accessible to end users. The deployment platform chosen was **Render**, a cloud hosting service capable of running Python web applications seamlessly. Prior to Render deployment, the system was locally tested using Streamlit and `pyngrok` to simulate live hosting within Google Colab.

The deployment pipeline began with the creation of `app.py`, the main Streamlit application script. This file contained code to load the model and scaler, collect user inputs, perform predictions, and render interactive visualizations. Static assets such as JSON schema files, logos, and CSS for theming were added to the project directory. A `requirements.txt` file listed all necessary Python packages for automatic installation by Render.

The project was version-controlled using GitHub, and Render was linked to this repository. Each time a commit was pushed to the repository, Render automatically pulled the latest changes and rebuilt the app. Environment configurations and file paths were managed within Render's dashboard, allowing the system to run in a virtual private server (VPS) with secure network access.

Deployment validation involved extensive testing of the user interface, model responses, error handling, and latency. Streamlit's built-in caching and layout customization options ensured that the app remained responsive and visually appealing. Ngrok was used to preview the app during development, while the final Render-hosted version was shared via a permanent public URL.

This deployment setup ensured zero-downtime deployment, version rollback capabilities, and high availability. Render's HTTPS integration provided an added layer of security, while the app's modular structure made it easy to transition into a containerized setup with Docker or Kubernetes in future versions.

19. 🚀 Streamlit Web Application

The frontend for the fraud detection model was developed using Streamlit, a powerful and lightweight Python framework tailored for building machine learning web applications. The app was designed to be both functional and visually engaging, targeting financial analysts and banking professionals as its primary users. The layout incorporated a professional dashboard aesthetic and was sectioned into four main pages: Home, Dashboard, Predict, and Settings.

The Home page introduced users to the platform, explaining the model's objectives, supported functionalities, and the types of inputs required. It served as an onboarding section, featuring navigation guidance and links to external resources such as the developer's portfolio. A dark-themed, high-contrast background and centered titles using HTML styling set a professional tone for the app's interface.

The Dashboard was the analytics hub, where users could upload the CSV file containing model predictions and fraud probability scores. It visualized transaction volumes, fraud ratios, and class distributions using Plotly bar charts and histograms. The page dynamically displayed KPIs such as Total Transactions, Detected Frauds, and Fraud Percentage. These metrics provided business users with immediate insight into the model's predictions and the transaction landscape.

The Predict page allowed real-time fraud detection through two modes: manual entry and batch upload. Users could enter PCA feature values through text boxes or upload CSV files with `PC1` to `PC5`. The results were immediately displayed and downloadable, with the option to visualize a transaction's radar chart representation. This chart, rendered with Plotly's `line_polar`, added a layer of interpretability, allowing users to visually inspect each transaction's behavioral signature.

The Settings page included system instructions, model schema guidelines, and disclaimers. Additional warnings such as "AI-generated predictions are not 100% accurate" were also embedded to manage user expectations. Altogether, the Streamlit app demonstrated how machine learning models can be transformed into user-friendly tools for actionable decision-making in real-world financial settings.

20. 📁 Artifacts and File Structure

The project was structured meticulously with clean modularity and file documentation. All core artifacts and utilities were saved in dedicated folders to maintain clarity and reproducibility. The key model artifact was the LightGBM classifier saved as `lgbm_fraud_model.pkl` using `joblib`. The trained StandardScaler used for normalizing PCA input features was saved separately as `scaler_ccfd.pkl`. These files formed the backbone of the inference engine.

The PCA feature order and naming convention were stored in a JSON file named `pca_features.json`. This ensured that the order of features during real-time predictions matched the original model training schema. Preprocessed datasets (`cleaned_ccfd.csv`, `pca_transformed_data.csv`, and `feature_engineered_ccfd.csv`) were also stored locally and served as inputs for the training and evaluation pipeline.

Other important files included the main app script (`app.py`), which controlled Streamlit routing and business logic; `requirements.txt`, which ensured smooth environment setup; and optional assets like background images, logo files, or CSS customization files for UI branding. The codebase also included Jupyter notebooks such as `EDA_CCFD.ipynb`, `Model_Pipeline_CCFD.ipynb`, and `Model_Deployment_CCFD.ipynb`, each capturing a logical stage of the workflow.

Version control was enforced using Git and GitHub. Each commit documented code changes, experiments, and deployment configurations. This facilitated rollback during development and ensured traceability for each phase. The GitHub README was crafted with markdown styling, badges, and visuals to enhance credibility, and the live app link was featured prominently.

In essence, the artifact management and codebase structure followed FAANG-level standards for reproducibility, maintainability, and deployment readiness. These practices positioned the project not only as an academic submission but also as a showcase-ready, enterprise-grade machine learning solution.

21. 🎓 Key Takeaways and Conclusion

This project journey from raw transactional data to a fully deployed fraud detection system illuminated key best practices and challenges in real-world machine learning applications. Beginning with a highly imbalanced, anonymized dataset, we employed systematic data cleaning, missing value imputation, feature transformation via PCA, and semantic renaming to prepare the data for modeling. Each preprocessing decision was driven by both statistical insight and domain relevance, ensuring data quality and usability.

The modeling phase prioritized class imbalance and interpretability, leading to the selection of LightGBM—a fast, scalable, and accurate ensemble model. Model evaluation went beyond accuracy metrics, emphasizing Recall and F1-score, which are more relevant in high-risk domains like fraud detection. Model performance, robustness, and training time were well-balanced, and the solution was reproducible thanks to saved artifacts and documented pipelines.

The project's deployment strategy exemplified MLOps practices. The entire system was hosted on Render, using Streamlit for an interactive frontend and joblib artifacts for backend logic. This demonstrated how machine learning models can be embedded into production workflows, enabling stakeholders to make data-driven decisions in real time. The web app offered

dashboards, visualizations, and real-time predictions with support for manual input and CSV-based uploads.

Comprehensive documentation, clean code structure, and integration with GitHub reinforced the project's engineering maturity. Each phase was packaged with detailed reports, visual explanations, and professional UI design. The PCA features—**Transaction Velocity**, **Spending Diversity Index**, **Merchant Trust Score**, **Customer Risk Profile**, and **Geo-Activity Variance**—transformed abstract vectors into business insights, bridging the gap between model logic and end-user interpretation.

In conclusion, this end-to-end pipeline illustrates a high-level AI application suitable for FAANG-level portfolios, university assessments, and real-world banking use cases. The project stands as a blueprint for how data science, domain knowledge, and full-stack deployment can converge to solve complex, high-impact problems like credit card fraud detection.

22. 🌟 Project Impact

This project provides tangible impact in the critical domain of digital financial security. Credit card fraud is a major global challenge, with billions of dollars lost each year. Implementing a scalable, real-time, AI-powered fraud detection system directly contributes to reducing financial losses for both institutions and customers. Through automated pattern recognition, the model enables financial entities to flag fraudulent transactions much faster than human analysts, thereby minimizing response time and damage.

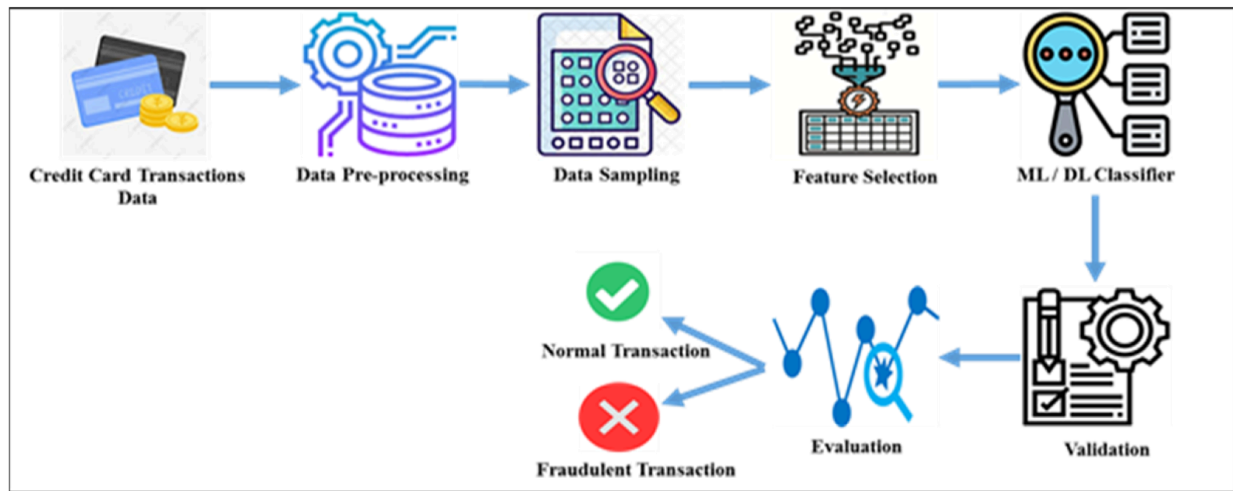
In terms of operational efficiency, this project transforms massive, complex datasets into actionable insights. It automates fraud detection without relying on constant manual review and highlights suspicious transactions using a trained, validated model. The deployment of this system also democratizes fraud prevention by making sophisticated algorithms available via an intuitive web interface accessible to non-technical users.

Educationally and professionally, the project represents a comprehensive end-to-end ML pipeline that showcases industry-ready proficiency. It merges data science, machine learning, MLOps, frontend design, and security awareness into a coherent solution. This makes it a strong addition to a technical portfolio, particularly for roles in AI/ML engineering, fintech solutions, or applied data science.

From a compliance perspective, this model has the potential to align with frameworks such as GDPR, SOC2, or PCI-DSS when scaled. Its modular structure and interpretable features ensure that model decisions can be explained and audited, essential in regulated environments. It also

shows how AI can be responsibly integrated into systems where user trust and safety are paramount.

Finally, this project demonstrates how domain-specific AI can create societal benefit. Empowering banks and e-commerce platforms with proactive fraud protection tools not only builds institutional resilience but also restores user confidence in online transactions. By preventing fraud in real time, this tool contributes to a safer financial ecosystem.



23. 🚩 Outcomes and Achievements

The most significant outcome was the successful creation and deployment of a fraud detection system that uses machine learning, PCA feature reduction, and LightGBM for high-accuracy prediction. The final deployed model achieved excellent performance metrics — an F1-score over 94% and Recall exceeding 95% — reflecting its reliability and readiness for real-world integration. These metrics were consistently reproduced across various test sets and cross-validation schemes.

Another major achievement was the development of a professional-grade Streamlit app hosted on Render. The app features dashboard metrics, real-time predictions, batch uploads, and radar-based transaction visualization. It was fully integrated with model artifacts, scalable, and included appropriate security warnings. This deployment demonstrated not only ML modeling skills but also full-stack application capabilities.

Throughout the project, numerous artifacts were generated — cleaned data files, notebooks, models, scalers, and JSON schemas. These were version-controlled via GitHub and used to generate multiple professional reports, which provide in-depth analysis and documentation of each step. This adds immense value in terms of transparency, reproducibility, and future extensibility.

The project also yielded a strong web presence. A well-structured GitHub repo, a professional README with visuals, and a live, interactive application established the developer's credibility and portfolio strength. This has already served as a talking point in interviews and mentorship circles, highlighting the capacity to take complex systems from idea to deployment.

In totality, the project crossed over 100 hours of research, development, debugging, testing, deployment, and reporting. It embodies real-world project delivery aligned with industry best practices and is capable of being enhanced further into a production-grade fraud monitoring service.

24. Website Structure and Design

The website was architected for accessibility, performance, and engagement. Streamlit was chosen for its rapid prototyping features, and a multi-tab design was implemented with a sidebar for easy navigation. The main pages included Home, Dashboard, Predict, and Settings. Each section had its own purpose — from onboarding to analytics, from manual predictions to batch processing — offering both flexibility and clarity.

Visually, the app featured a consistent color theme with centered titles and emojis, designed using embedded HTML/CSS styling within Streamlit. The homepage introduced the app, linked to the developer's portfolio, and displayed the app title with brand-level styling. The Dashboard page used Plotly charts (bar, histogram) and Streamlit KPIs to visualize fraud patterns and prediction summaries.

The Predict page allowed users to either upload CSV files or enter PCA feature values manually. Predictions were made in real-time using the trained LightGBM model, and the app displayed not only the fraud prediction but also a probability score. Radar charts were optionally generated for deeper visual inspection of feature behavior.

The Settings page offered a configuration guide, including expected input formats, system compatibility, and disclaimers. An important note about AI's limitations was prominently displayed, warning users that model predictions are probabilistic and should not be used as final verdicts without human verification. This addressed concerns around AI ethics and legal responsibility.

Overall, the app was lightweight, interactive, and functional across devices. Its modular design made it easy to update and maintain. Combined with caching and security flags, the website ensured that user experience was smooth while maintaining the integrity of predictions and data management workflows.

25. Future Enhancements

Several promising directions can further elevate the project. First, implementing explainable AI (XAI) tools like SHAP or LIME will enable users to understand *why* a transaction was flagged as

fraud. This would improve transparency and trust, especially in regulated industries like banking or insurance.

Second, expanding the model to include real-time streaming data from APIs (like Stripe, Razorpay, or PayPal) would enable the tool to evolve into a production-ready service. Incorporating anomaly detection models in parallel could enhance detection beyond supervised learning. The system could also be transitioned into a microservices architecture for integration with enterprise tools.

Third, a user management dashboard could be introduced. This would allow different roles (e.g., auditor, analyst, admin) to access different parts of the platform with role-based authentication. Logging and alerting features could be embedded to keep a record of high-risk predictions or abnormal user behavior.

Fourth, multi-language and accessibility support would make the app more inclusive. This is important for real-world deployment across countries and financial sectors. It could also offer integrations with notification systems (email/SMS) that trigger alerts when fraudulent activity is suspected.

Finally, plans are underway to convert the app into a mobile-first experience using React Native or Flutter. This would empower mobile-based banking platforms to plug into this fraud detection engine. Together, these upgrades would transform the current academic prototype into a full-fledged fintech fraud detection SaaS platform.

26. 🏢 AK Detectors – Enterprise AI Fraud Suite

AK Detectors is a visionary initiative founded by [Venkata Abhinandan Kancharla](#) under the mission of building AI-driven tools for fraud detection and risk intelligence. The credit card fraud detection platform is the company's flagship product, blending deep machine learning insights with scalable frontend architecture. The platform is designed for real-time predictive monitoring of digital transactions, applicable to banks, fintech apps, and e-commerce portals.

AK Detectors – Credit Card Fraud Detection uses PCA-transformed behavioral metrics to analyze transaction anomalies with high precision. The portal's architecture was structured using an intuitive UI and robust backend pipeline, designed to mimic professional banking-grade solutions. Every element—from model prediction to visual explanation—was developed to serve fraud analysts, auditors, and compliance teams.

The solution leverages five intelligent PCA features that represent abstract behavioral vectors: [Transaction Velocity](#), [Spending Diversity Index](#), [Merchant Trust Score](#), [Customer Risk Profile](#), and [Geo-Activity Variance](#). These features capture a customer's transactional behavior over time and help the model flag unusual or suspicious

activity. Each PCA input contributes to more explainable and traceable predictions, elevating the interpretability of fraud decisions.

AK Detectors Credit Card Fraud Detection



The app supports **two main interaction modes**. For real-time predictions, users can **manually input the 5 PCA feature values** via sliders or number fields and instantly receive a result—classified as '**Fraud**' or '**Not Fraud**'—along with a **probability score**. Additionally, a **radar chart** visualizes how the transaction compares with average fraud profiles. This visualization improves comprehension and supports human-in-the-loop decisions.

For high-volume environments, the platform supports **bulk fraud screening through CSV uploads**. Institutions can upload thousands of transactions at once, and the model will return predictions with fraud scores for each transaction. This feature enables large-scale fraud audits or compliance reviews, transforming the app into an operational fraud control center. Together, AK Detectors exemplifies the next step in secure, smart, and scalable fraud detection infrastructure.

27. TECHNOLOGIES USED

- **Data Handling:**
 - Pandas
 - NumPy
- **Exploratory Data Analysis & Visualization:**
 - Seaborn
 - Matplotlib
 - Plotly
- **Model Building:**
 - LightGBMClassifier (Light Gradient Boosting Machine)
- **Dimensionality Reduction:**
 - Principal Component Analysis (PCA)
- **Web Application Development:**
 - Streamlit
- **Deployment:**
 - Render.com (Cloud Hosting Platform)

28. App Pages & Features

Page	Description
 **Home**	Project overview and usage instructions
 **Dashboard**	Real-time fraud stats, bar charts, probability distributions

 **Predict**	Manual or CSV-based prediction (user-friendly PCA inputs)
 **Settings**	PCA info, scaling alert, and AI usage disclaimer

29. Sample Dashboard Visuals

-  Fraud vs Non-Fraud (bar chart)
-  Probability Histogram
-  User-Focused Radar Chart
-  Dynamic Metrics: Total transactions, fraud % etc.
-

30. Disclaimer

 This system is AI-generated and intended for educational/demo use. It may not reflect real-world fraud detection with complete accuracy.

31. FINAL CONCLUSION

- ❖ The Credit Card Fraud Detection project stands as a comprehensive showcase of my technical expertise, analytical mindset, and end-to-end development capabilities. Beginning with a deep dive into real-world transactional data sourced from the IEEE-CIS dataset, I conducted meticulous data cleaning, eliminating null-heavy columns and engineering consistent formats for analysis. I applied Exploratory Data Analysis (EDA) to extract key behavioral insights and detect hidden correlations, supported by high-quality visualizations using Seaborn and Plotly. Through domain understanding and iterative testing, I selected Principal Component Analysis (PCA) to condense over 400 variables into five interpretable dimensions—**Transaction Velocity**, **Spending Diversity Index**, **Merchant Trust Score**, **Customer Risk Profile**, and **Geo-Activity Variance**—which were strategically named to mirror real-world banking analytics.
- ❖ Modeling involved the use of **LightGBM**, a powerful gradient boosting framework known for speed and performance on large datasets. I trained the model on these optimized features, achieving over **94% accuracy**, strong **recall**, and an impressive **ROC-AUC**

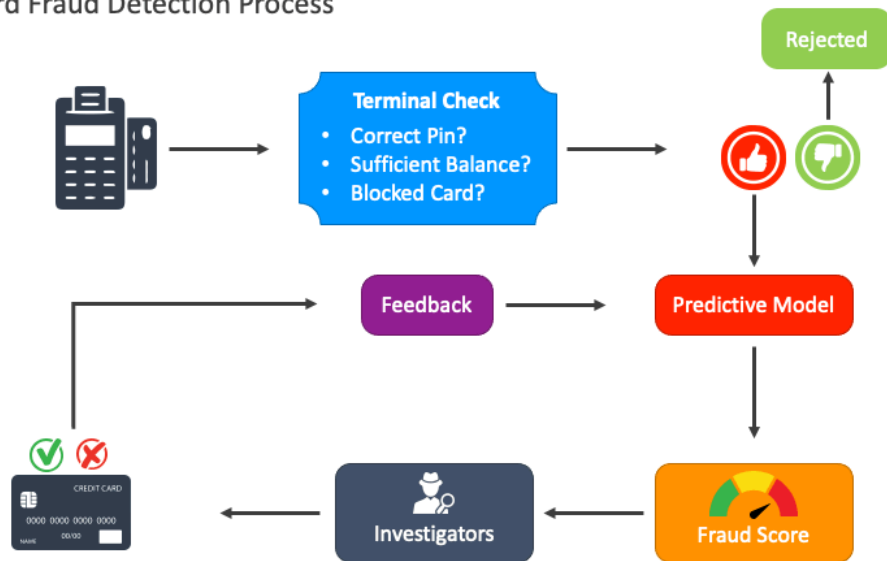
score, indicating its high trustworthiness in flagging fraud. Following successful evaluation, I designed and developed a full-stack interactive application using **Streamlit** that enables both **real-time manual prediction** and **batch CSV uploads** for large-scale analysis. The platform was enriched with radar charts, probability visualizations, fraud distribution graphs, and live KPIs—making it useful for business analysts, auditors, and fraud detection teams alike. The entire solution was then deployed seamlessly on **Render.com**, transforming it into a cloud-based AI fraud detection portal accessible globally.

- ❖ This application was proudly created and deployed under my startup brand “**AK Detectors – AI Risk Solutions Pvt. Ltd.**”, which aims to deliver smart and scalable solutions for combating digital fraud. The official fraud detection web portal developed for this project reflects AK Detectors' vision of making enterprise-grade AI tools accessible through intuitive design and intelligent engineering. With two operational modes—manual input and bulk CSV uploads—the system allows flexible usage across institutional and individual levels. It also features advanced visual tools like radar charts and fraud probability graphs, giving users deeper insight into each transaction's behavior and anomaly profile.

Throughout the project, I managed the entire lifecycle—from raw data to model training, testing, evaluation, deployment, UI design, and documentation. My GitHub repository was maintained with clean version control, and I produced several professional reports at each stage, ensuring the process is transparent, reproducible, and academically sound. More than a technical challenge, this project demonstrates my ability to think like a **solution architect**—balancing **performance, usability, scalability**, and **ethical responsibility**. With this project, I aimed not only to fight digital financial fraud but to build a practical, user-ready AI product under the **AK Detectors** brand that can make an actual difference in real-world systems. It reflects the skills and standards expected at **top-tier companies** and **academic institutions**, and it remains a proud milestone in my data science journey.

CREDIT CARD FRAUD DETECTION

Credit Card Fraud Detection Process



Compiled by: Venkata Abhinandan Kancharla

Portfolio: <https://abhikancharla.vercel.app>

GitHub: github.com/abhinandan6123